

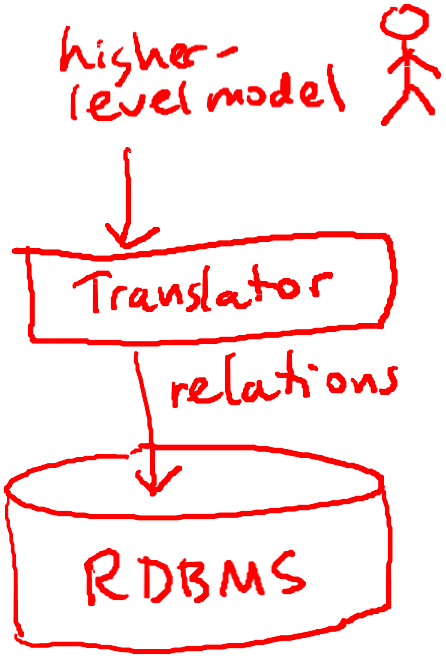
UML - Unified Modeling Language
Use of UML for Designing Database

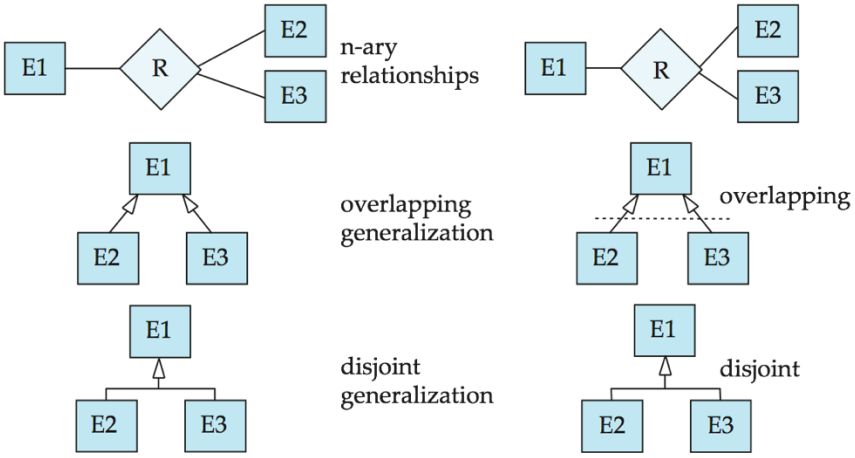
Assoc. Prof. Kitsana Waiyamai, Ph. D.
Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



Higher-Level Database Design Models (Conceptual Level)

- **Diagrams** are more formal than **Natural Languages**
- **Diagrams** are user-friendly graphical specification language
- **E/R** (Entity-Relationship Model) is quite old
- **UML** (Unified Modeling Language)
  => Data modeling **subset**
- Both are graphical & can be translated to relations
  - automatically or semi-automatically





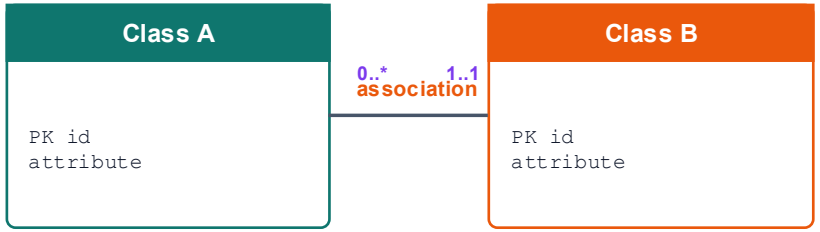
UML – Unified Modeling Language

- Database design starts before CREATE TABLE.
- UML helps us communicate the real-world structure.
- UML is what we draw for humans; relations are what we build for the DBMS.
- Every useful UML model must eventually become relations and constraints.

```
Requirements
     ↓
UML Conceptual Schema
     ↓
Relational Schema
     ↓
Tables + Constraints
+ SQL
```

UML – Unified Modeling Language

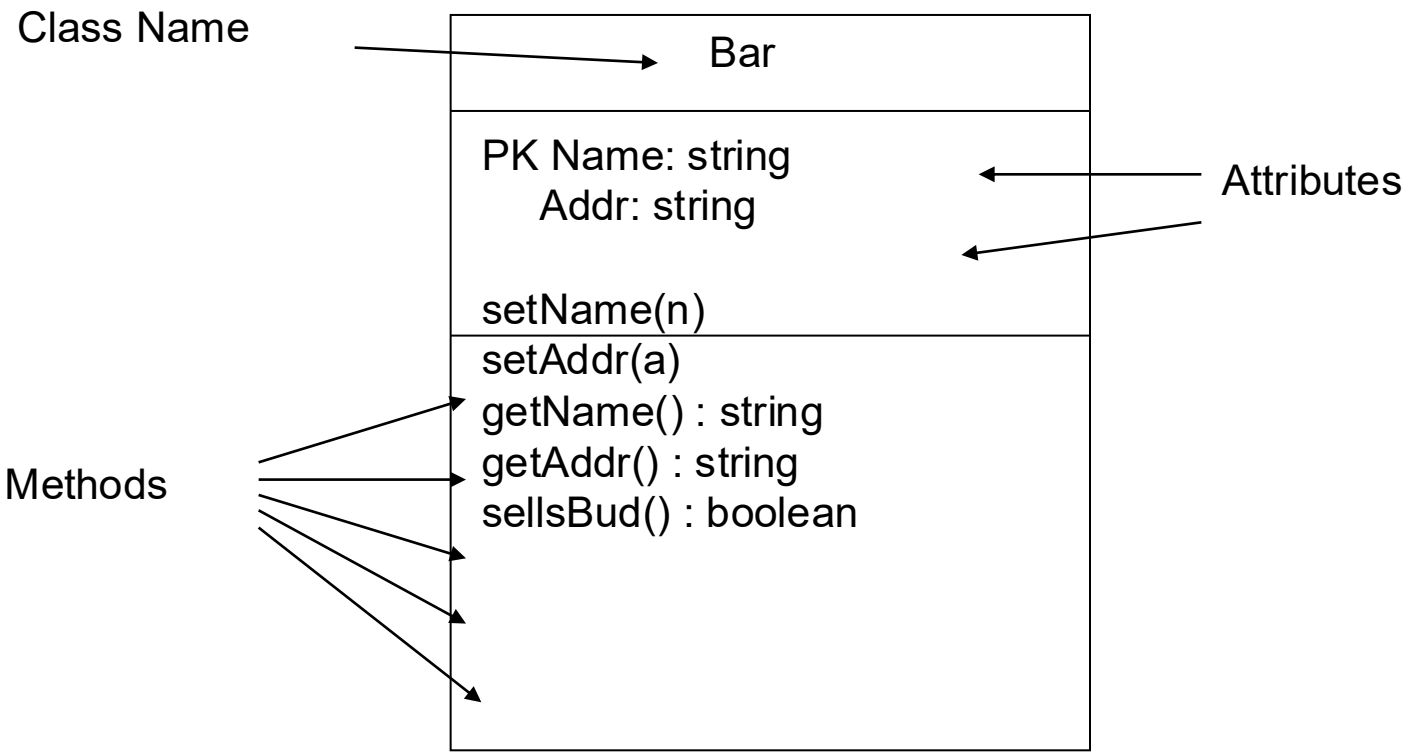
- UML is designed to model software but has been adapted as a database modeling language.
- UML is a conceptual design *language* that helps us think and communicate before creating tables.
- UML Data Modeling: 5 concepts
  - Classes
  - Associations
  - Association Classes
  - Super-classes & Sub-classes
  - Composition & Aggregation



- Every UML diagram we draw must eventually become tables.

Class

- **Class** = set of similar objects with attributes (*state*) and methods (*behavior*).
- **Attributes** represent state and have types.
- PK indicates an attribute in the primary key of the object.
- Methods have declarations: arguments (if any) and return type.
- **Example**: Bar class

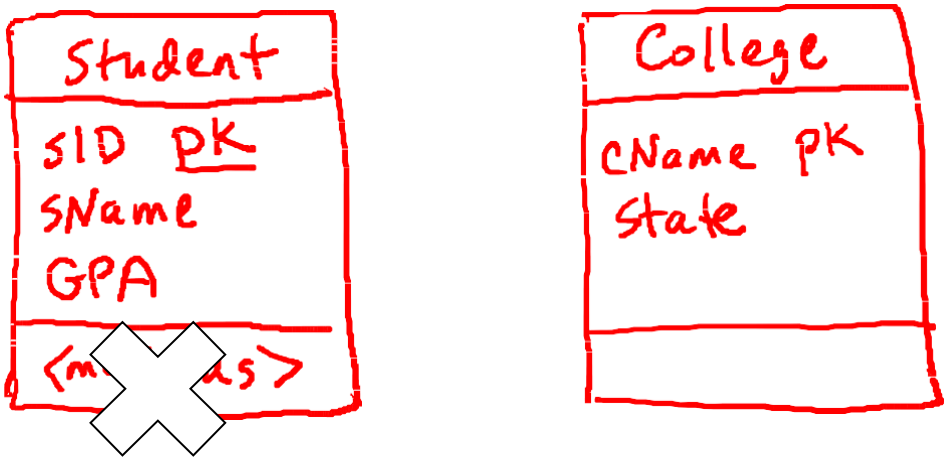


UML Data Modeling

Class

Name, attributes, methods

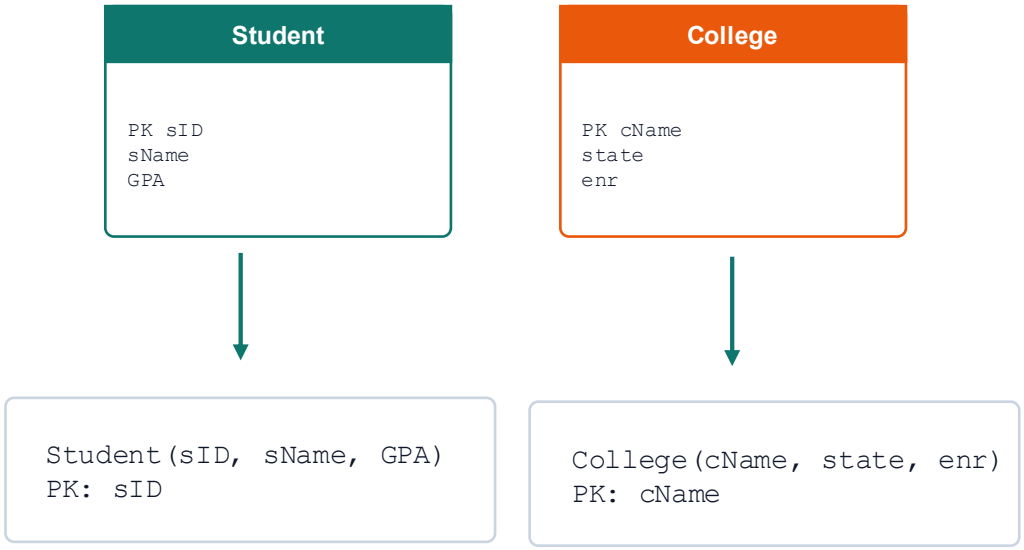
For database modeling: ***drop*** *methods*



UML Class

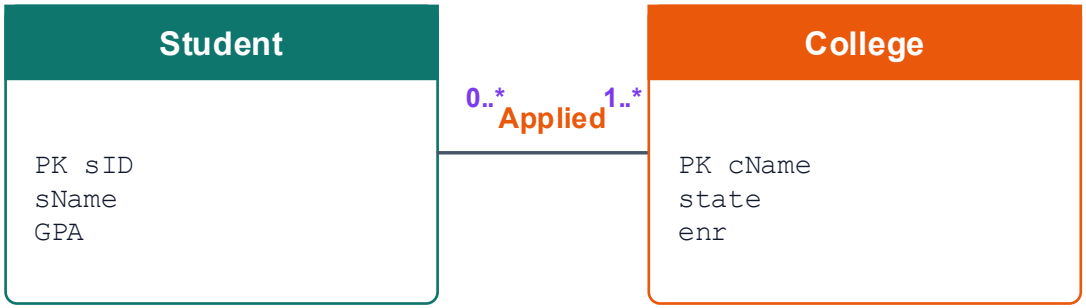
Later, each UML class becomes one relation

- Class name → Table name
- Attributes → Columns
- PK → Primary key



UML Modeling

Association



- Association = relationship between objects of two classes.
- Represented by a named line between classes.
- The name should be a meaningful verb or noun phrase.

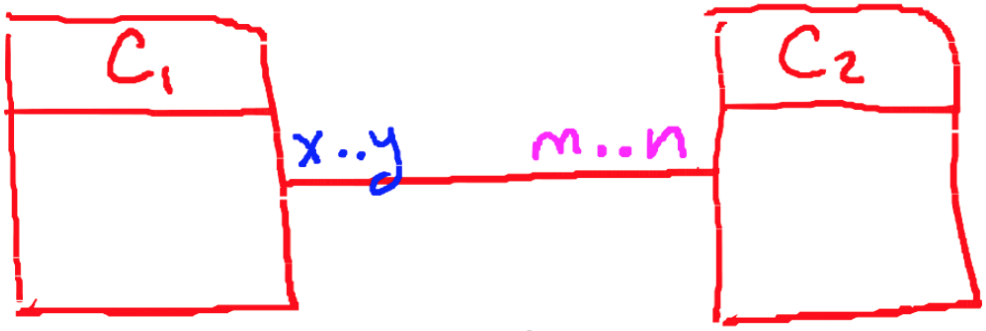
UML Modeling

Multiplicity of Associations

Relationships between objects of two classes

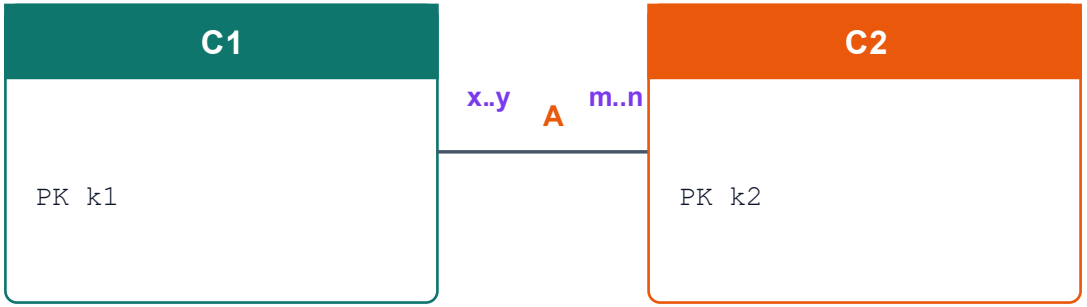
- Each object of class C_1 is related to at least m and at most n objects of class C_2

- Each object of class C_2 is related to at least x and at most y objects of class C_1



Multiplicity Notation

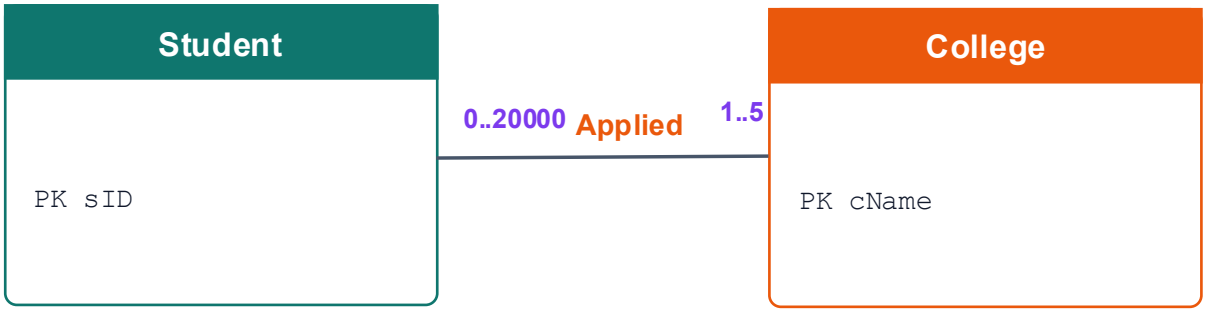
- *m* ..*n*
  - means between *m* and *n*,
  - at least m and at most n
- *
  - means “infinity”
  - many / unbounded
- 0..1
  - optional, at most one
- 1..1
  - mandatory exactly one
- 1..*
  - means “at least one”
- 0..*
  - means “arbitrary number”,
  - no restriction



UML Modeling

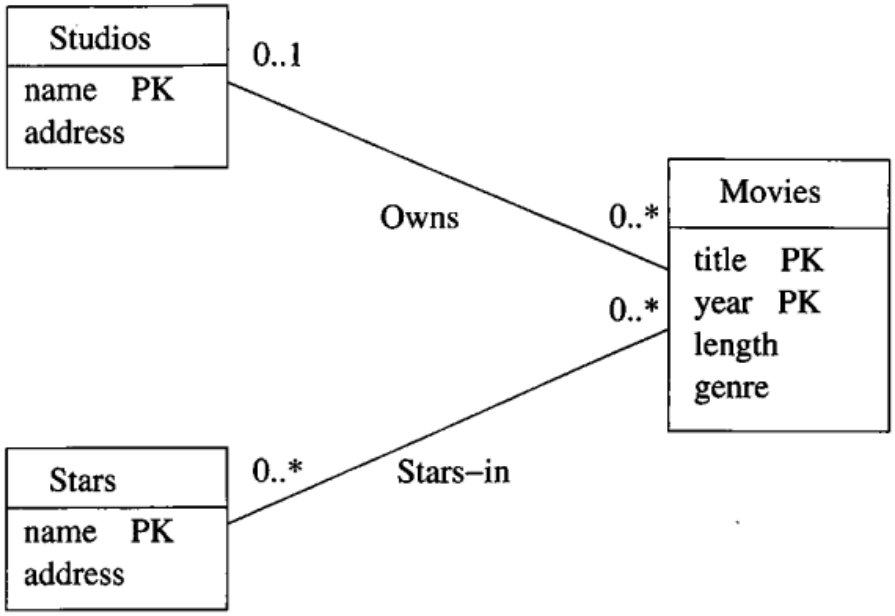
Multiplicity of Associations

- Students must apply somewhere and may not apply to more than 5 colleges.
- No college takes more than 20,000 applications.



Examples: Association





Types of Relationships

Multiplicity of Associations: Types of Relationships

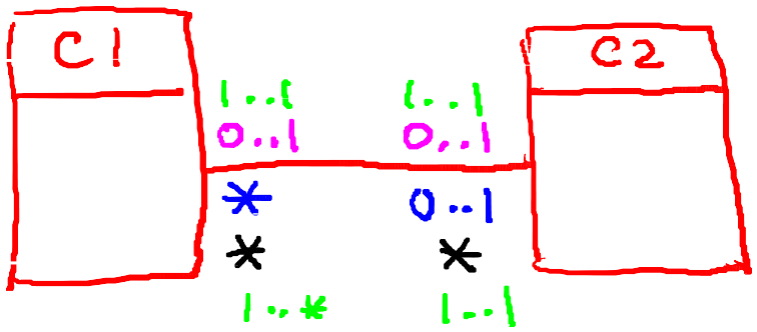
- One-to-One => 0..1 -- 0..1 (Optional One-to-One)
- Many-to-One => 0..n -- 0..1
- Many-to-Many => 0..* -- 0..* => * -- *
- **Complete Relationships** => Each object must participate in a relationship
  - **Complete** => Each object must participate in a relationship
  - Complete One-to-One => 1..1 -- 1..1
  - Complete Many-to-One => 1..* -- 1..1
  - Complete Many-to-Many => 1..* -- 1..*

- Complete participation → minimum is 1
- Optional participation → minimum is 0
- **Default => 1..1 – 1..1 => Complete One-to-One Relationship**

UML Modeling

Multiplicity of Associations: Types of Relationships

- One-to-One
- Many-to-One
- Many-to-Many
- Complete



Default 1..1

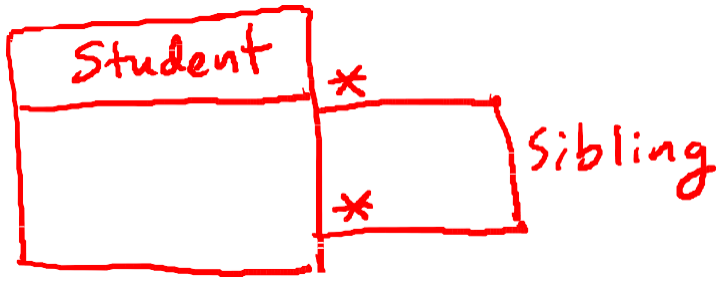
** UML Modeling: Exercise-I **

** UML Modeling: Exercise-II **

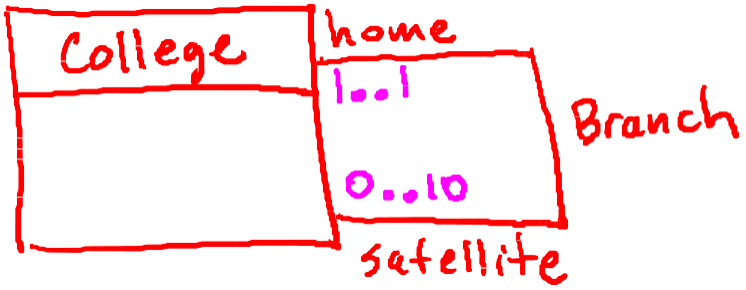
UML Modeling

Self-Associations

Associations between a class and itself



- Every student has arbitrary number of siblings

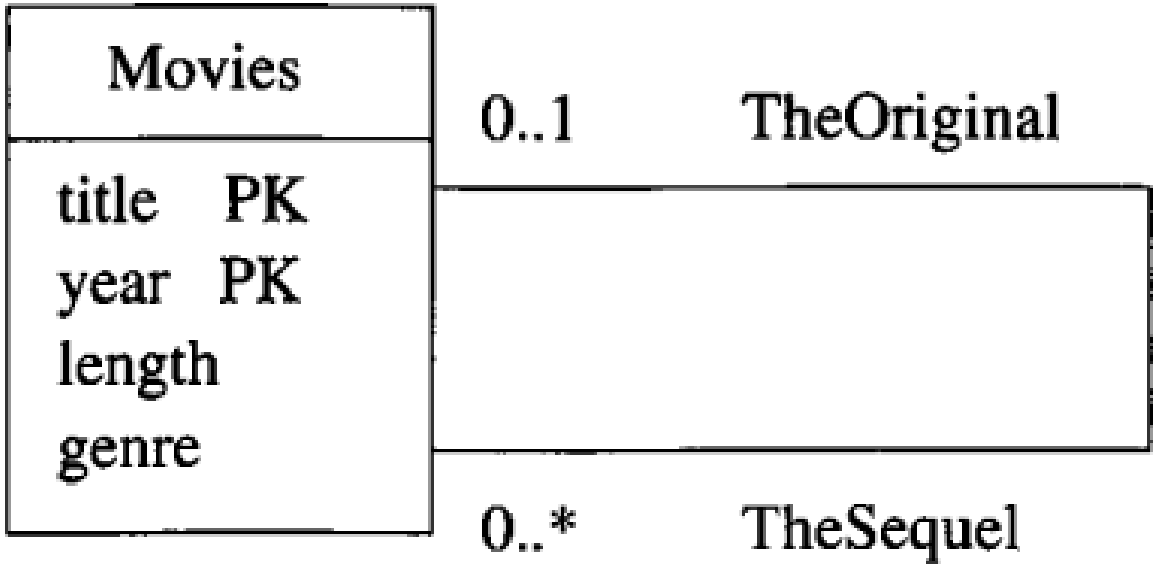


- One college is a branch of another college
- Every satellite campus must have one home campus
- Every home campus can have until 10 satellites

UML Modeling

Self-Associations

Associations between a class and itself

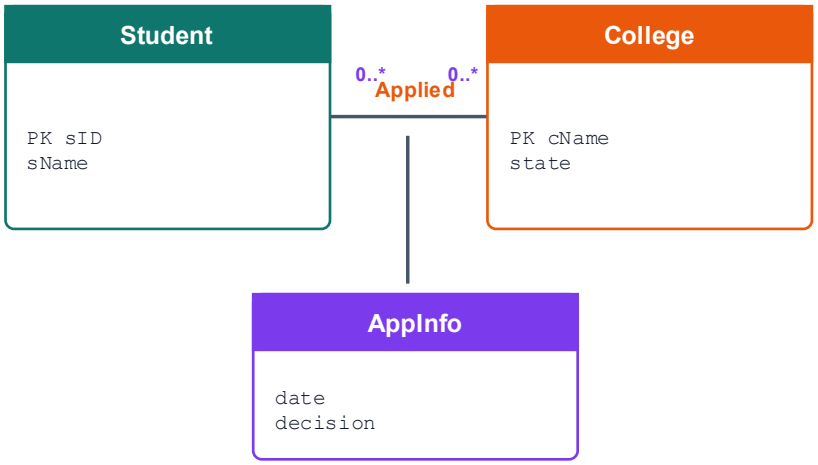


Self-association representing sequels of movies

UML Modeling

Association Classes

Relationships between objects of two classes,
with attributes on relationships



- Student applies to College.
- Each application has its own information: date and decision.
- That information belongs to the association instance, not just Student or College.

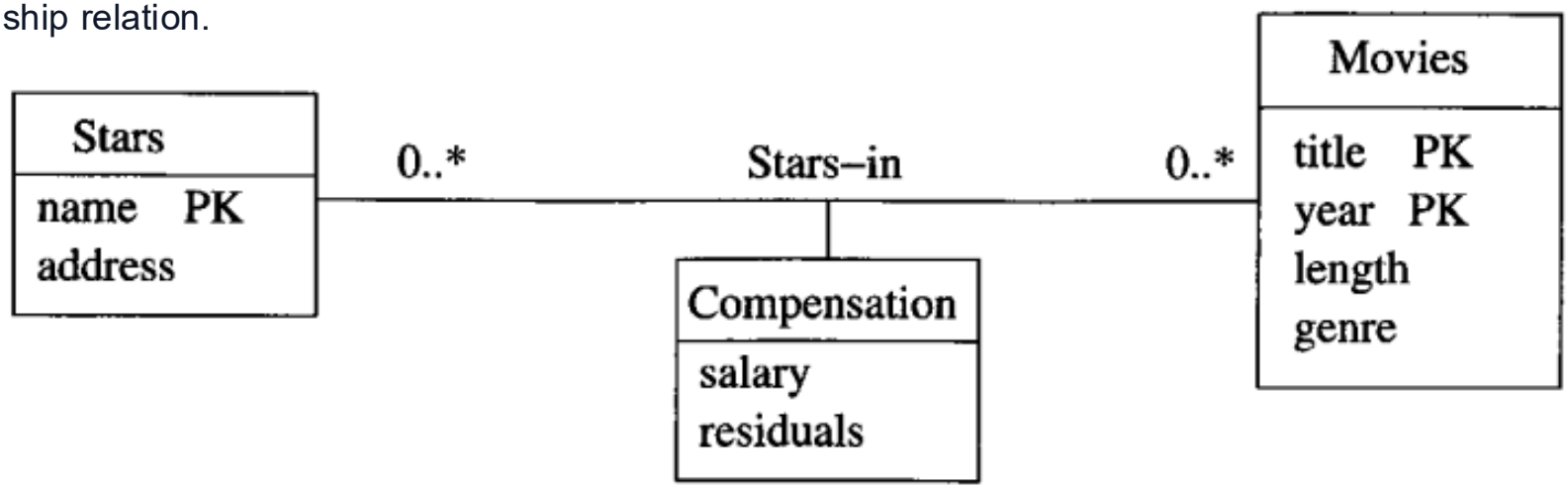
Like a class, but attached to an association

$\Rightarrow$ Each instance of the association between Student & College has additional information (**date, decision**)

UML Modeling

Association Classes

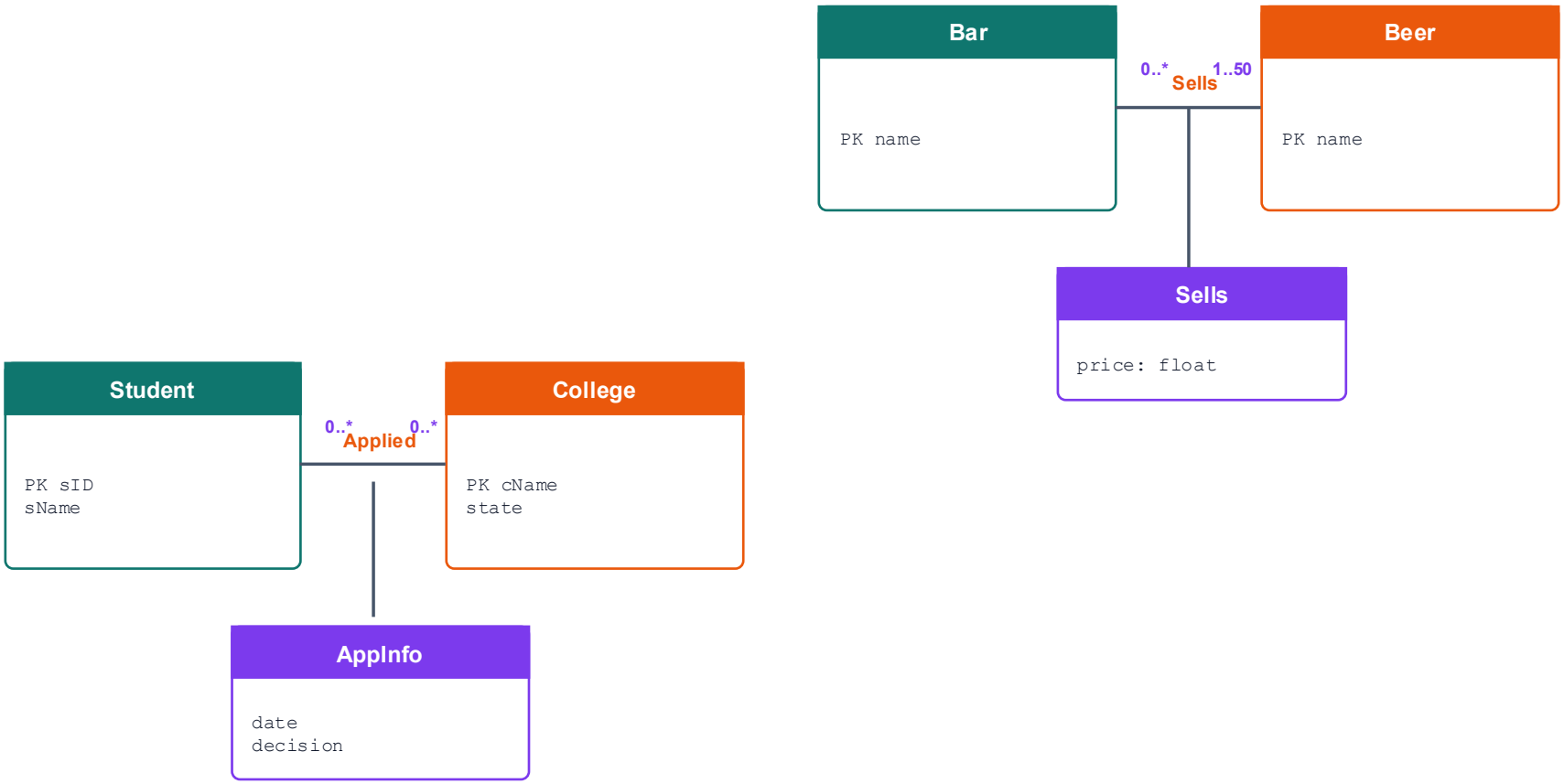
- Stars-In is M:N between Stars and Movies.
- Compensation belongs to each star-movie pair.
- salary and residuals become attributes in the relationship relation.



Compensation is an association class for the Association "Stars-in"

Association Class

Association-class attributes usually become columns in the relationship table.



** UML Modeling: Exercise-III **

UML Modeling

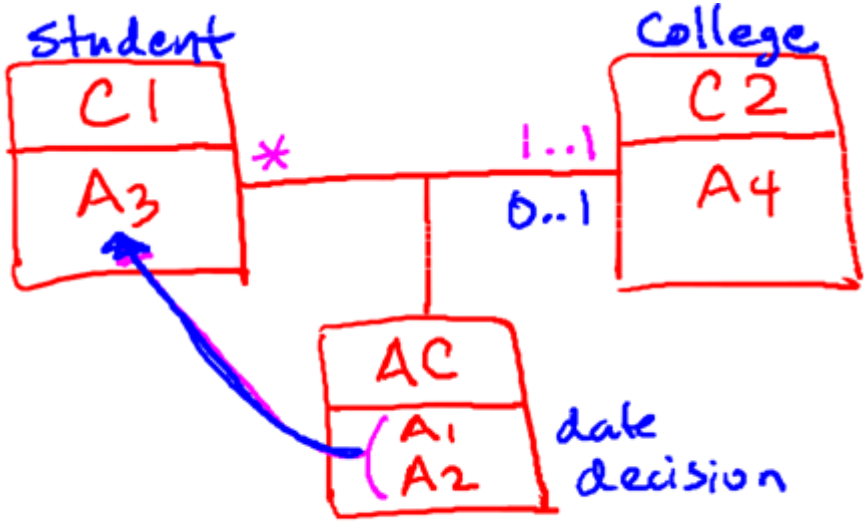
Eliminating Association Classes

Unnecessary if 0..1 or 1..1 multiplicity

UML Modeling

Eliminating Association Classes

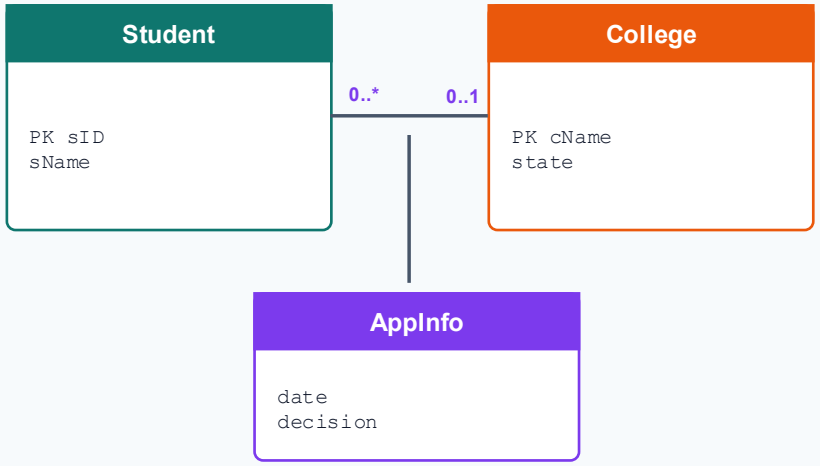
Unnecessary if 0..1 or 1..1 multiplicity



UML Modeling

Eliminating association classes

- If one side has multiplicity 0..1 or 1..1, **an association class may be unnecessary => Its attributes** can sometimes be folded into the many-side relation.
- This is an implementation choice, not a diagramming rule.



Before:

```
Student(sID, ...)
College(cName, ...)
Applied(sID, cName, date, decision)
```

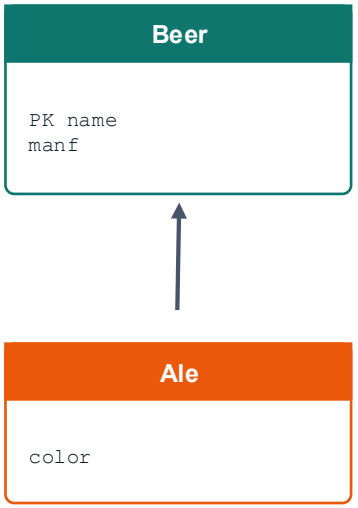
After (folded example):

```
Student(sID, ..., cName, date, decision)
College(cName, ...)
```

UML Modeling

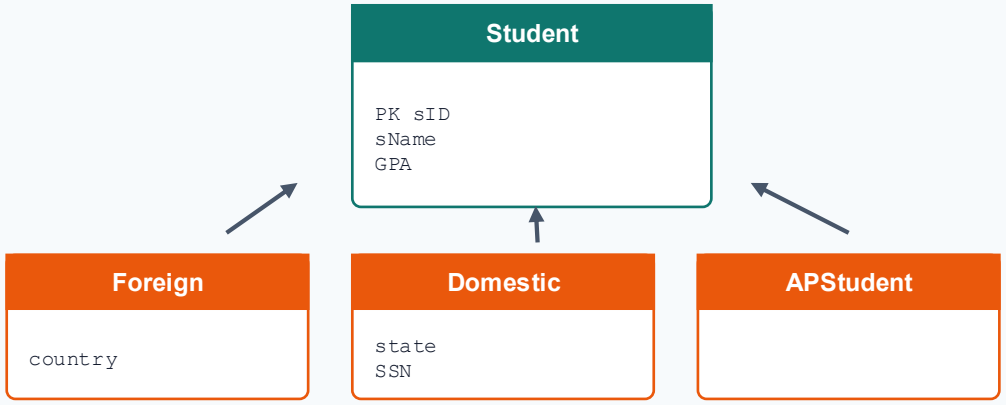
UML Data Modeling: **Super-classes, Sub-classes**

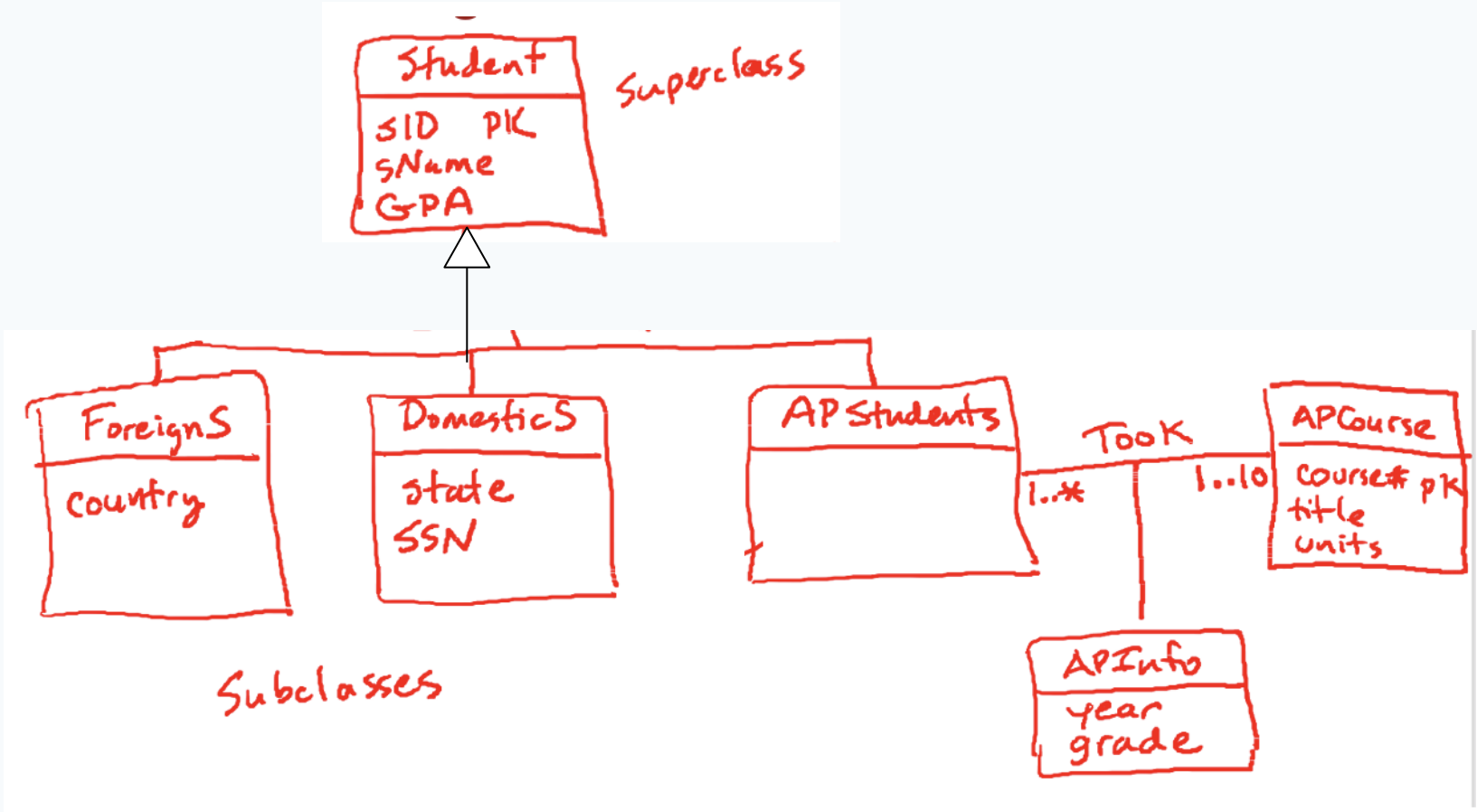
- Superclass = general type (*Generalization*)
- Subclass = specialized type (*Specialization*)
- Subclass inherits properties from its superclass.
- Used when objects share some attributes but differ in others.



Student superclass example

Student is the superclass.
Foreign, Domestic, and APStudent are subclasses.
Subclasses add specialized attributes or relationships.





UML Modeling

Subclass Terminology & Properties

- *Incomplete* (*Partial*) vs. *Complete*
  - *Is every object in super-class is in at least one sub-class?*
  - *If so, the subclasses are complete*
  - *Otherwise, they are incomplete (partial)*
- *Disjoint* (*Exclusive*) vs. *Overlapping*
  - *Every object is in at most one sub-class => Disjoint*
  - *Object can be in multiple sub-classes => Overlap*
- *Combination is possible !*

Subclass properties

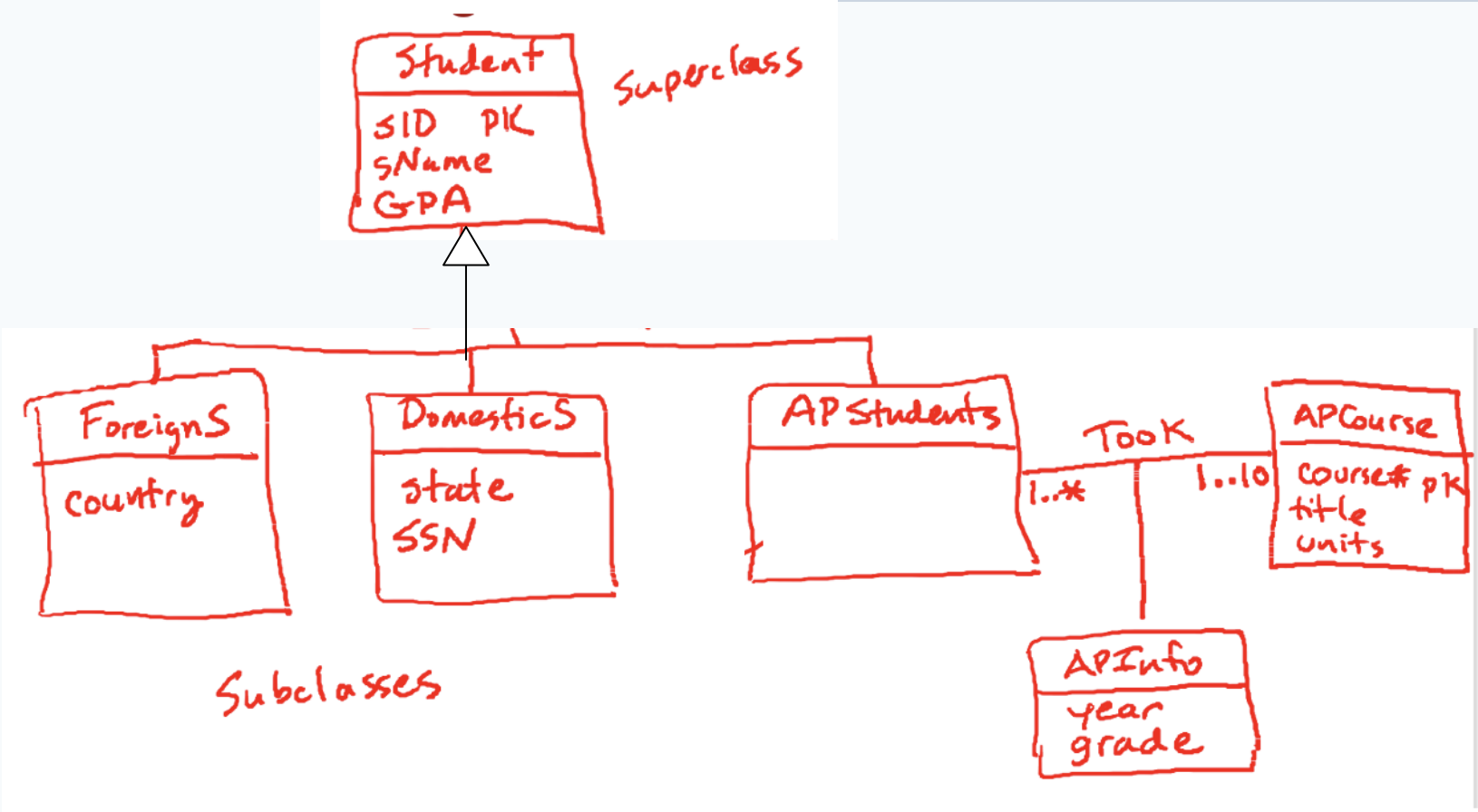
- Complete vs partial: must every superclass object belong to a subclass?
- Disjoint vs overlapping: can an object belong to more than one subclass?
- These properties affect the best relational translation.

Complete: every Student is either Foreign, Domestic, ...

Partial: some Students may be neither.

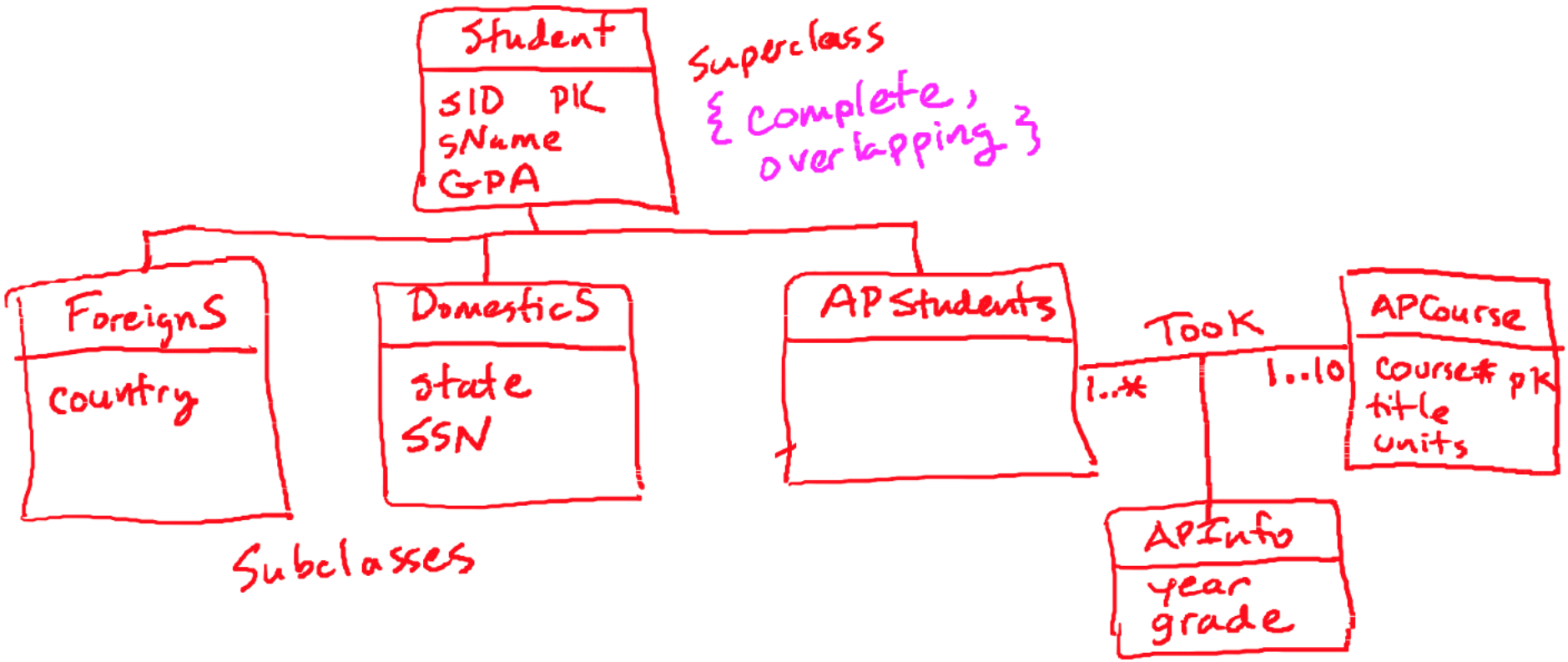
Disjoint: at most one subclass.

Overlapping: may belong to multiple subclasses.



UML Modeling

Super-classes, Sub-classes



UML Modeling

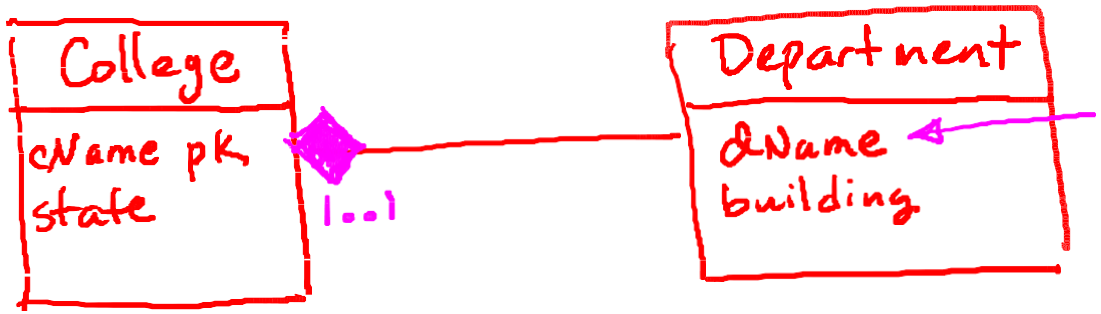
Composition & Aggregation

- Objects of one class **belong to** objects of another class.
- A special type of association.

UML Modeling

UML Data Modeling:
Composition

- Objects of one class belong to objects of another class
- The part is strongly dependent on the whole.
- Each department belongs to one college (1 .. 1)



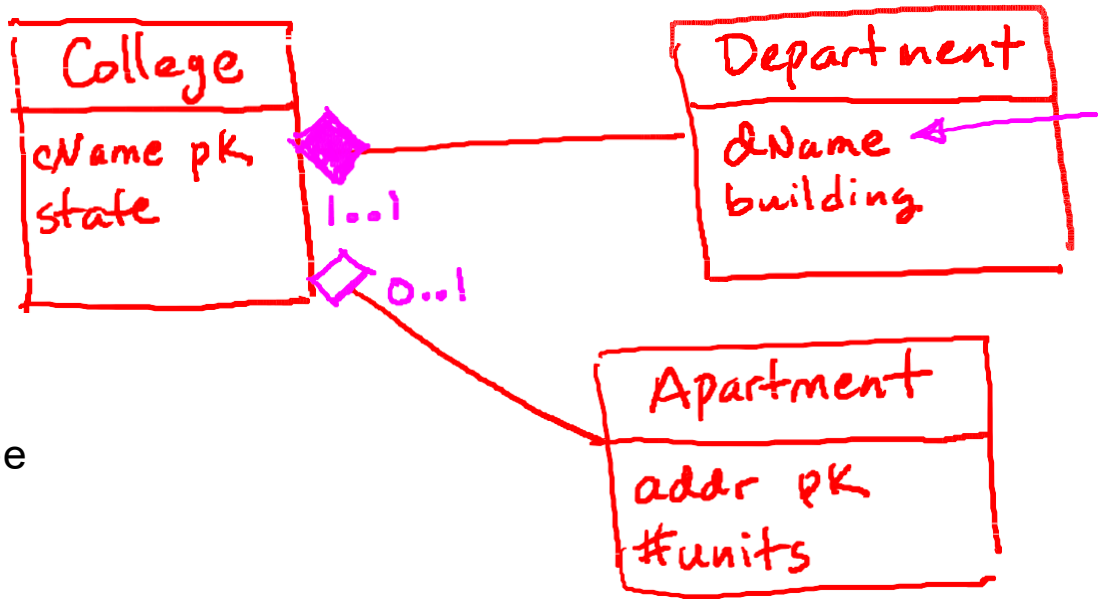
Order ◆——— OrderItem

House ◆—— Room

UML Modeling

Composition & Aggregation

- Objects of one class **belong to objects** of another class
- Aggregation is **weaker** and the child may exist without that parent.
- **Composition:** Each department belongs to one college (1 .. 1)
- **Aggregation:** Each apartment belongs to zero or one college (0 .. 1)



Department ◇——— Employee
University ◇—— Professor
Car ◆—— Engine

UML Modeling

Higher-Level Database Design

- *Unified Modeling Language* (UML)

  Data modeling subset

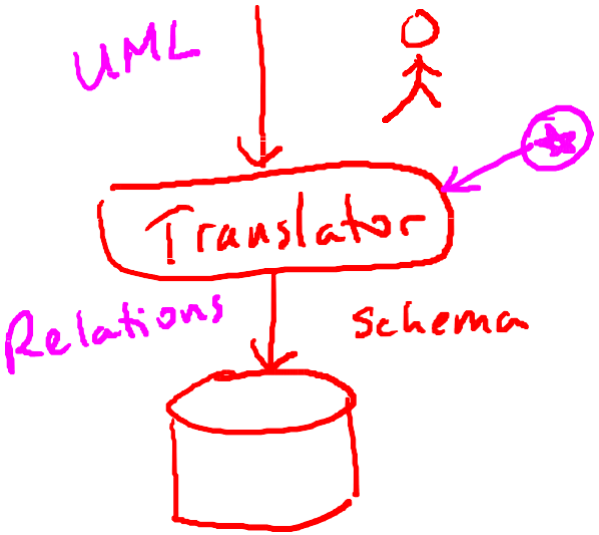
- Graphical
- 5 concepts
  (1) Classes
  (2) Associations
  (3) Association Classes
  (4) Super-classes-Sub-classes
  (5) Composition & Aggregation

❖ Can be translated to relations automatically

UML to Relations

High-Level Database Design Model

- User-friendly (graphical) specification language
- Translated into model of DBMS



UML to Relations

Classes => Relations

For each Class, create a Relation

=> Name = Name of the Class

$\Rightarrow$ Attributes are the attributes of the class.

- ## Associations => Relations.

For each association, create a relation with

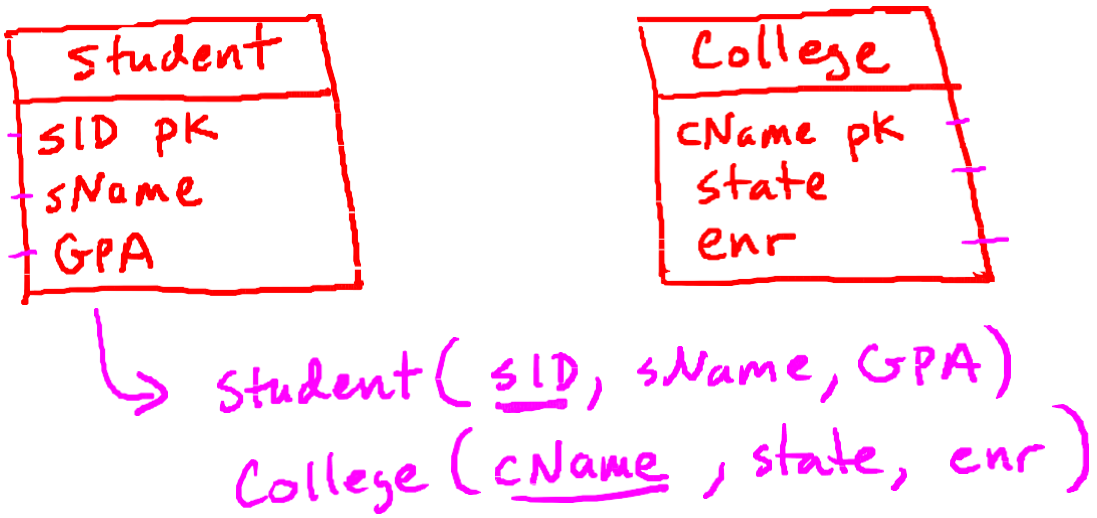
=> Name = Name of the Association

=> Attributes = Key(s) of attributes of the two classes.

=> If there is a coincidence of attributes between the two classes, rename them appropriately.

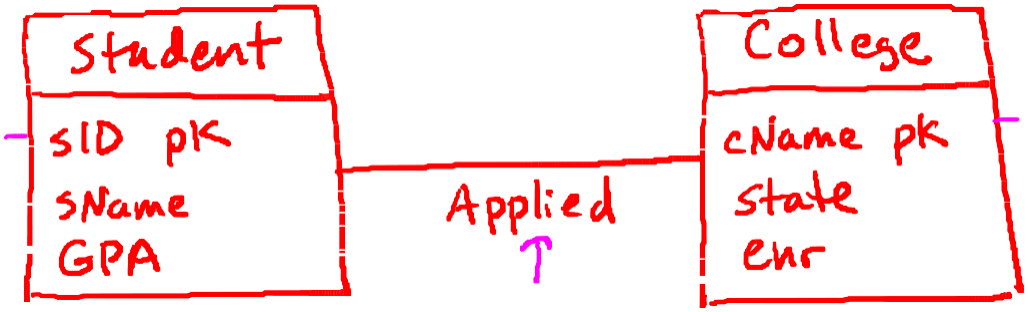
UML to Relations: **Classes**

Every class becomes a relation; pk → primary key



UML to Relations: **Associations**

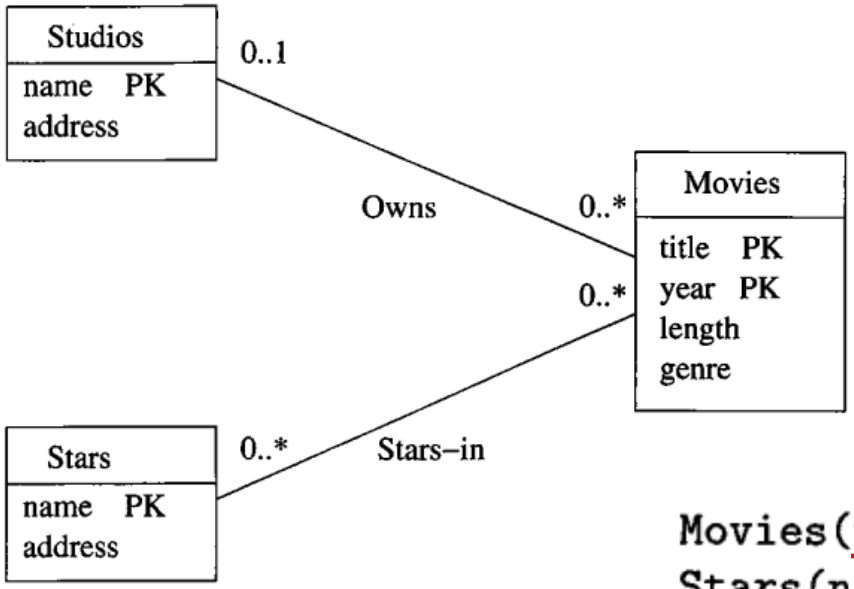
- ✓ Relation with key from each side
- ✓ The **default key** in the relation generated from an association is the **combination of the PKs** from the two classes in the association.



- Student(...)
- College(...)

Applied (sID, cName)

UML to Relations: Associations




```
Movies(title, year, length genre)
Stars(name, address)
Studios(name, address)
```

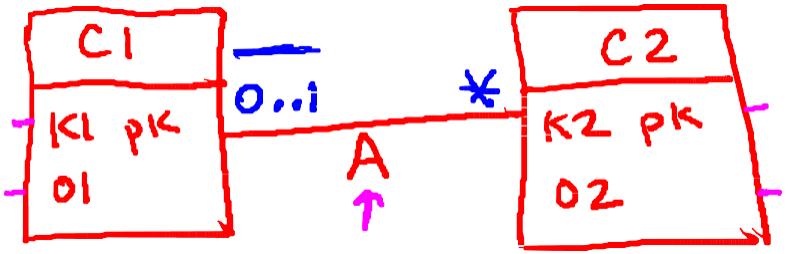
For the two associations, we create relations

```
Stars-In(movieTitle, movieYear, starName)
Owns(movieTitle, movieYear, studioName)
```

From UML to Relations

Keys for Association Relations

When the association is many-one, the key can be made even smaller by **eliminating the PK from the *one* side of the many-one association**.



C1(K1, O1)

C2(K2, O2)

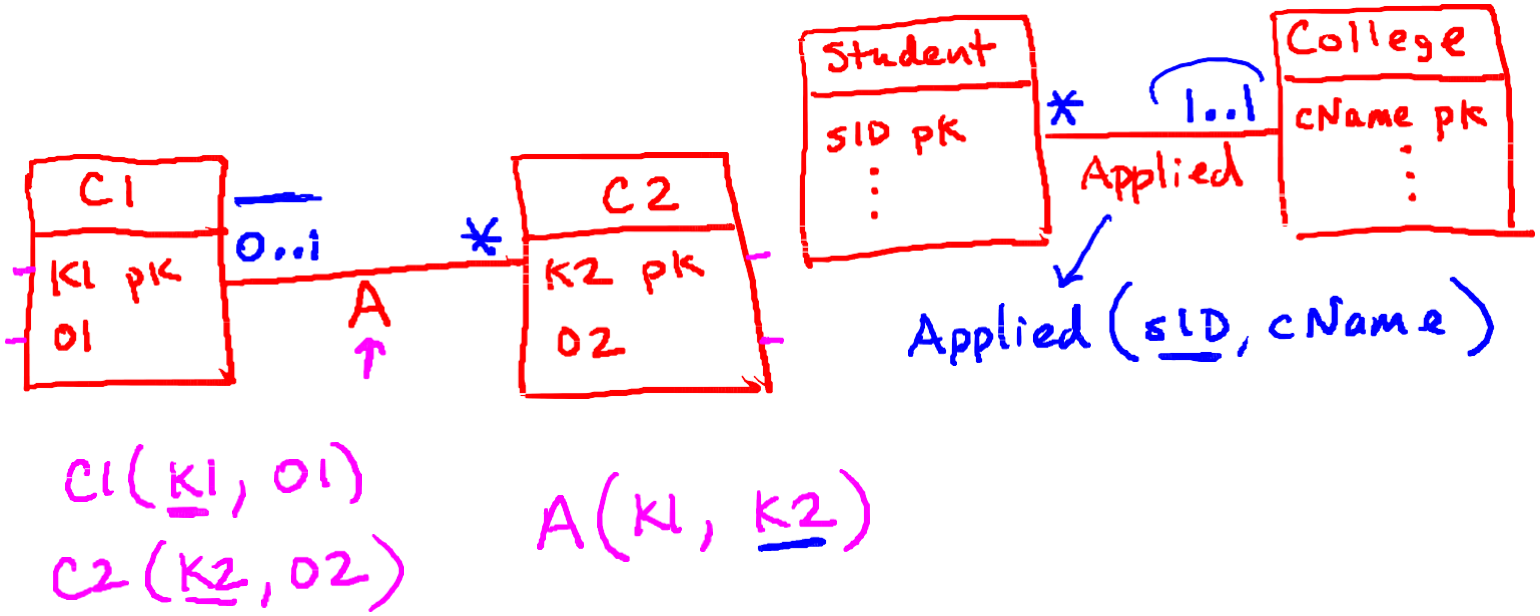
A(K1, K2)

If one side is 0 .. 1 or 1 .. 1,

then the other side is the key for the association relation

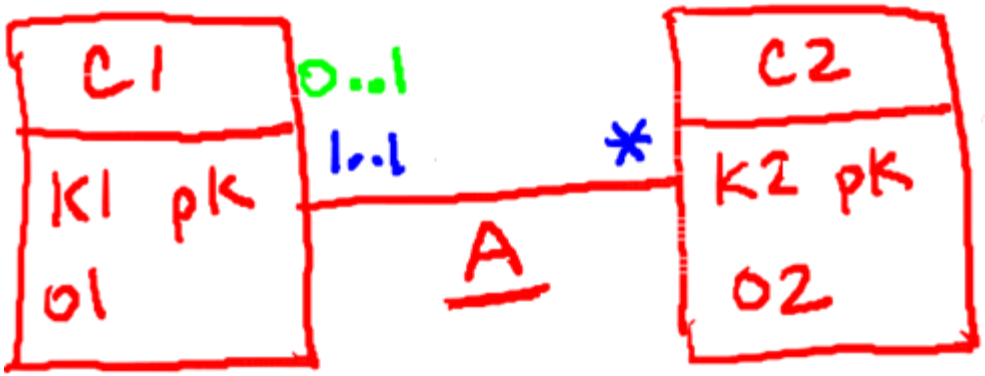
Keys for Association Relations

When the association is many-one, the key can be made even smaller by **eliminating the PK from the *one* side of the many-one association**.



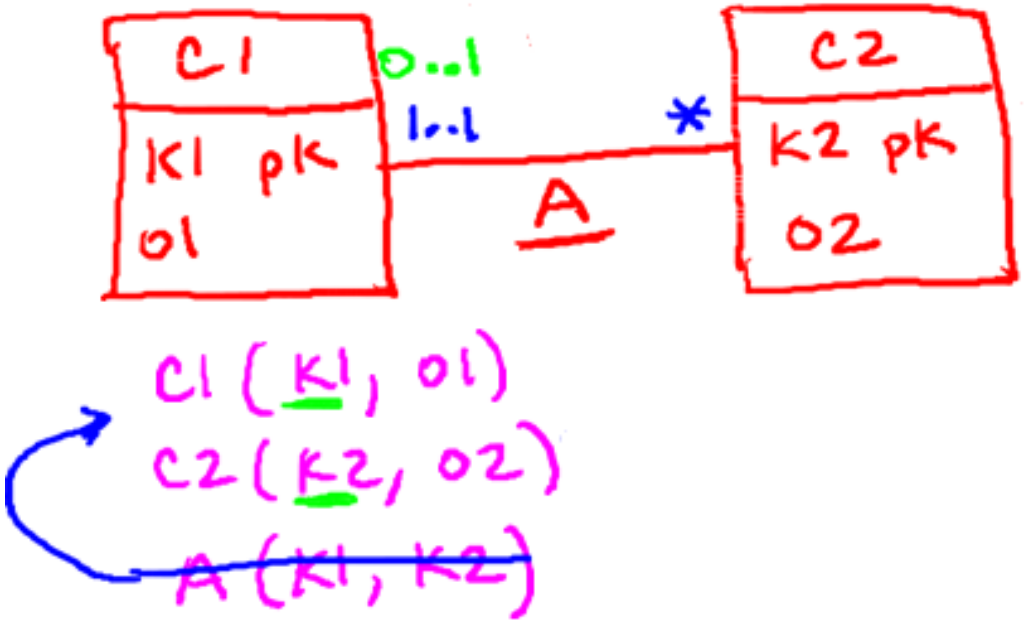
Association Relation Always Needed?

Depends on multiplicity



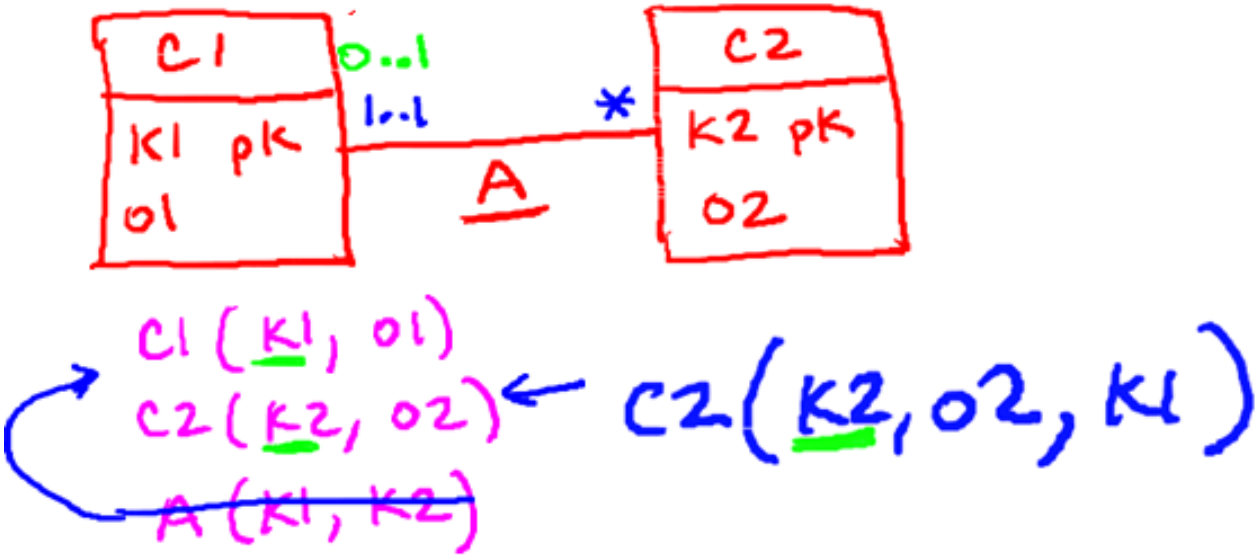
Association Relation Always Needed?

Depends on multiplicity



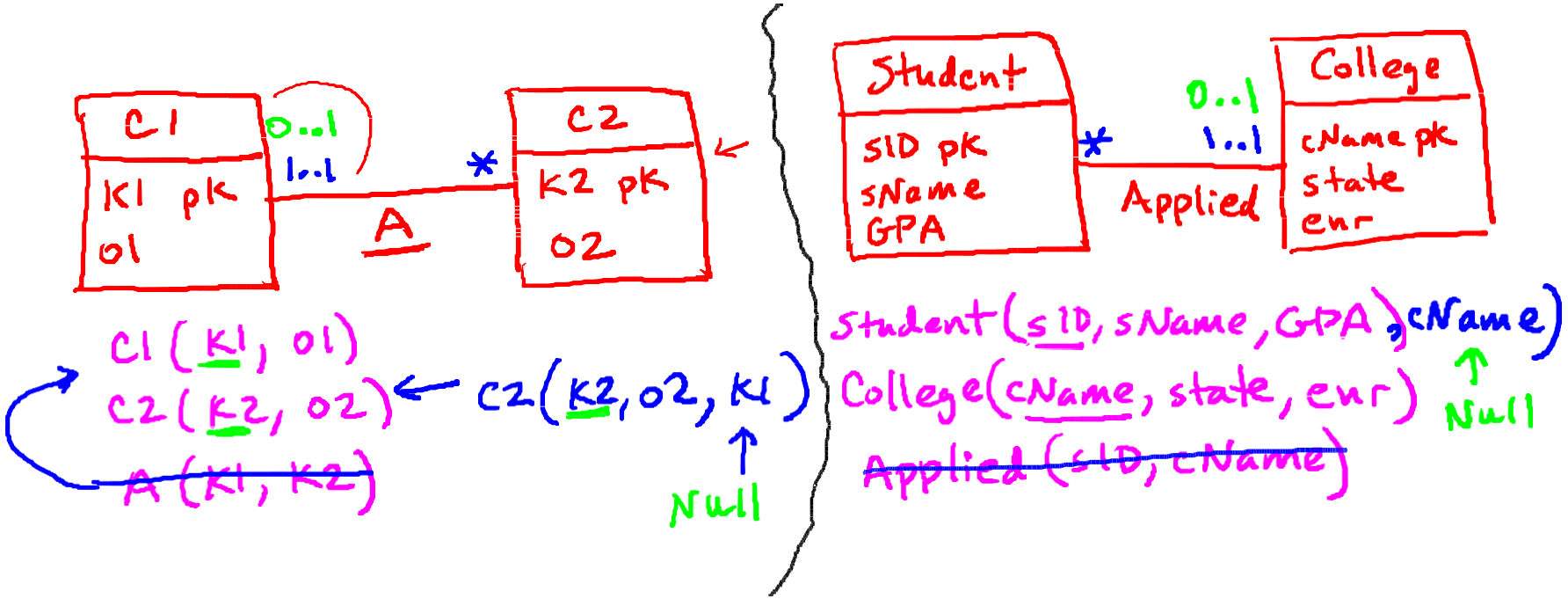
Association Relation Always Needed?

Association can be folded, it depends on multiplicity

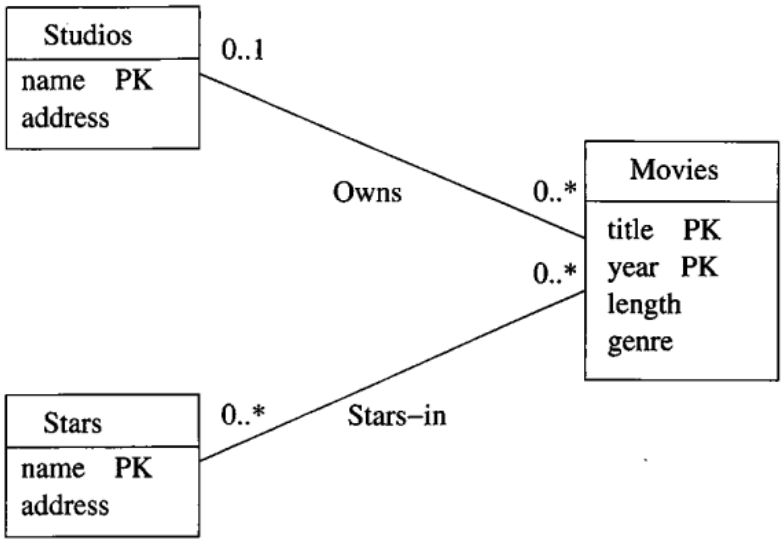


Association Relation Always Needed?

Depends on multiplicity



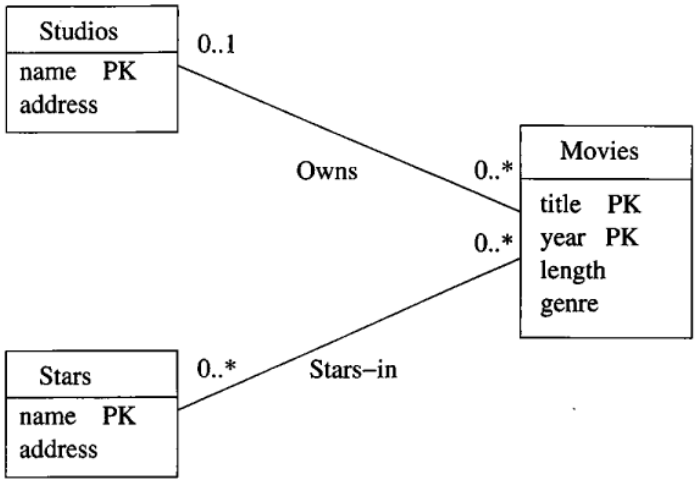
UML to Relations: Associations




```
Movies(title, year, length, genre, studioName)
Stars(name, address)
Studios(name, address)

Stars-In(movieTitle, movieYear, starName)
```

UML to Relations: Associations



Movies(title, year, length, genre, studioName)
Stars(name, address)
Studios(name, address)

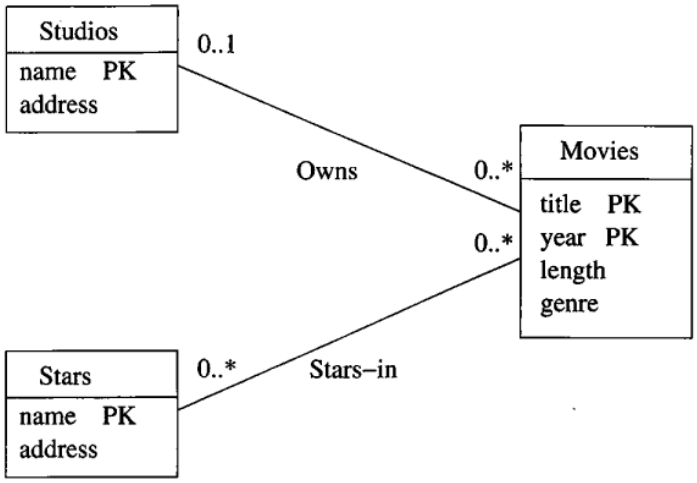
Stars-In(movieTitle, movieYear, starName)

```
-- Studios Entity Table
CREATE TABLE Studios (
    name VARCHAR(100) PRIMARY KEY,
    address VARCHAR(255)
);

-- Movies Entity Table
CREATE TABLE Movies (
    title VARCHAR(200),
    year INTEGER,
    length INTEGER,
    genre VARCHAR(50),
    studioName VARCHAR(100),
    PRIMARY KEY (title, year),
    FOREIGN KEY (studioName) REFERENCES Studios(name)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

-- Stars Entity Table
CREATE TABLE Stars (
    name VARCHAR(100) PRIMARY KEY,
    address VARCHAR(255)
);
```

UML to Relations: Associations




```
-- Table for Many-to-Many relationship (Stars-in)

CREATE TABLE StarsIn (
    movieTitle VARCHAR(200),
    movieYear INTEGER,
    starName VARCHAR(100),
    PRIMARY KEY (movieTitle, movieYear, starName),

    -- Foreign key constraints
    FOREIGN KEY (movieTitle, movieYear) REFERENCES
Movies(title, year),
    FOREIGN KEY (starName) REFERENCES Stars(name)
);
```

Movies(title, year, length, genre, studioName)
Stars(name, address)
Studios(name, address)

Stars-In(movieTitle, movieYear, starName)

UML to Relations: Associations

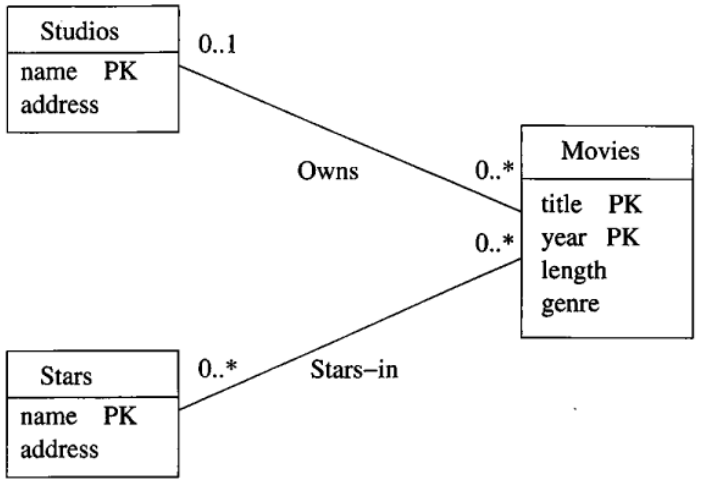
```
CREATE TABLE Studios (
    name VARCHAR(100) PRIMARY KEY,
    address VARCHAR(255)
);

CREATE TABLE Movies (
    title VARCHAR(200),
    year INTEGER,
    length INTEGER,
    genre VARCHAR(50),
    studioName VARCHAR(100),
    PRIMARY KEY (title, year),
    FOREIGN KEY (studioName) REFERENCES Studios(name)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

CREATE TABLE Stars (
    name VARCHAR(100) PRIMARY KEY,
    address VARCHAR(255)
);

CREATE TABLE StarsIn (
    movieTitle VARCHAR(200),
    movieYear INTEGER,
    starName VARCHAR(100),
    PRIMARY KEY (movieTitle, movieYear, starName),
    FOREIGN KEY (movieTitle, movieYear) REFERENCES Movies(title, year)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (starName) REFERENCES Stars(name)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

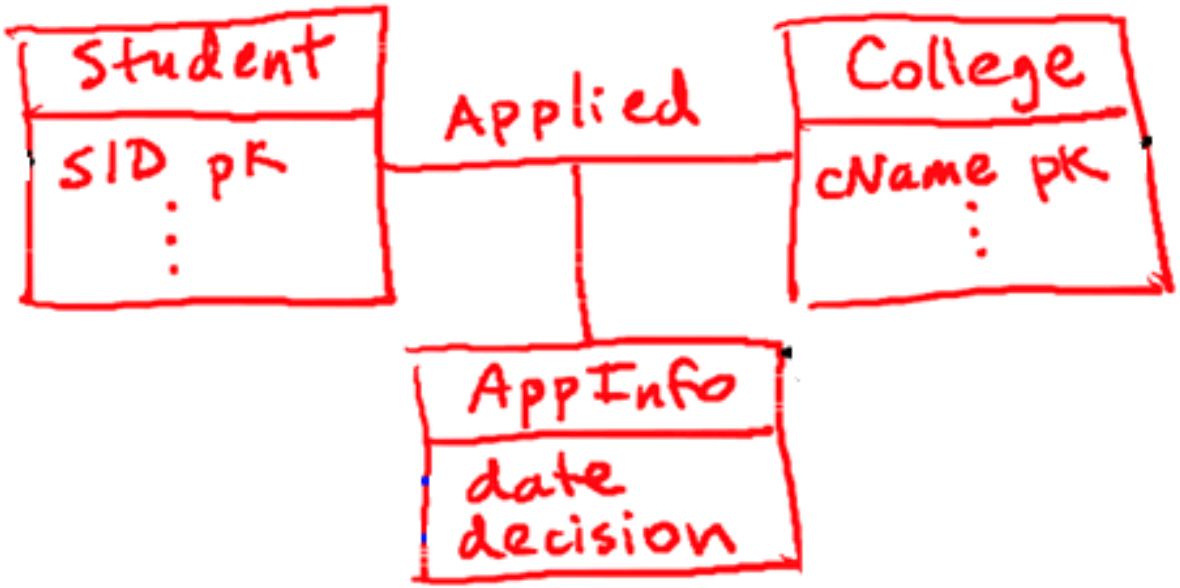
```
-- Create indexes for better query performance
CREATE INDEX idx_movies_studio ON Movies(studioName);
CREATE INDEX idx_starsin_movie ON StarsIn(movieTitle, movieYear);
CREATE INDEX idx_starsin_star ON StarsIn(starName);
```



** UML Modeling: Exercise-IV **

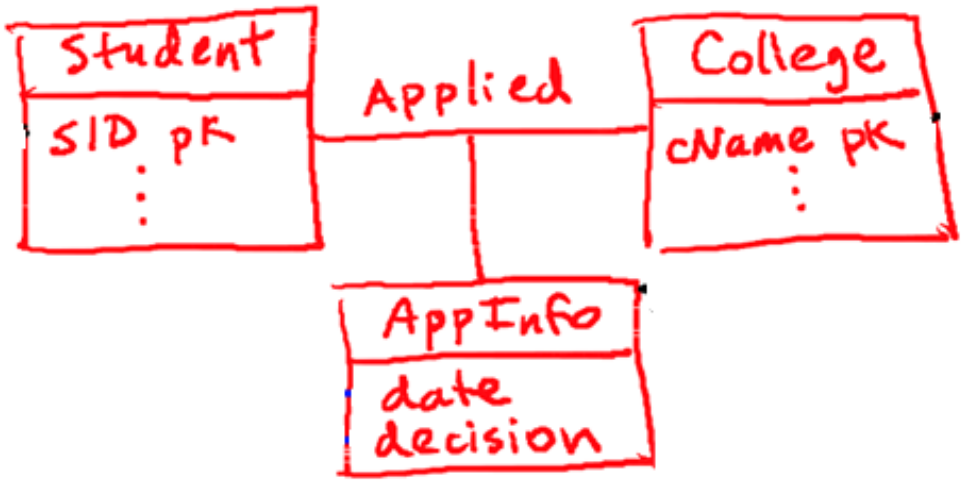
UML to Relations: **Association Classes**

Add attributes to relation for association



UML to Relations: **Association Classes**

Add attributes to relation for association



Student(sID, ...)

College(cName, ...)

Applied(sID, cName, date, decision)

Can still use folding rule !

UML to Relations: **Association Classes**

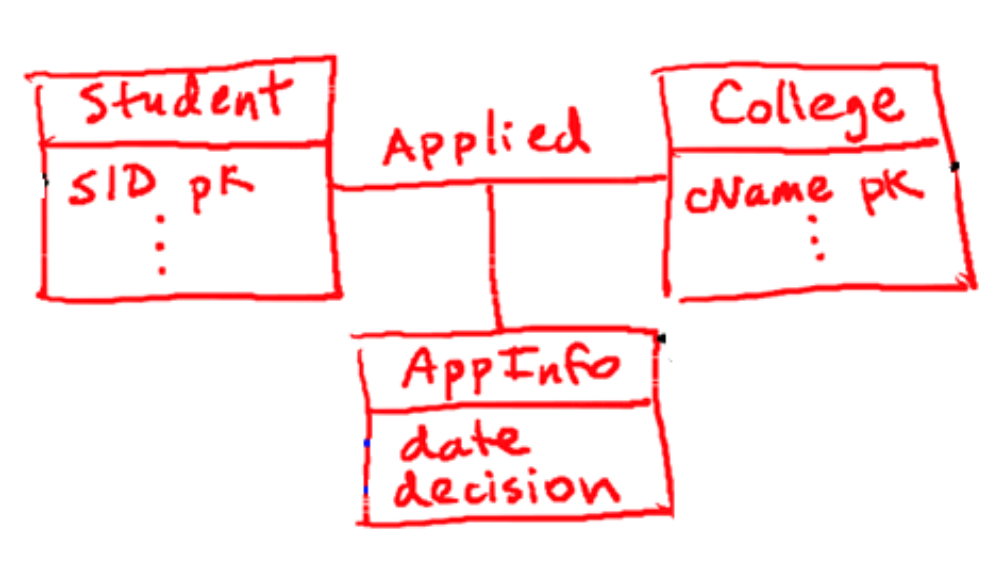
Add attributes to relation for association

- Create the association relation as usual.
- Add the association-class attributes as columns.
- Usually the PK includes the participating keys.

```
Student(sID, ...)

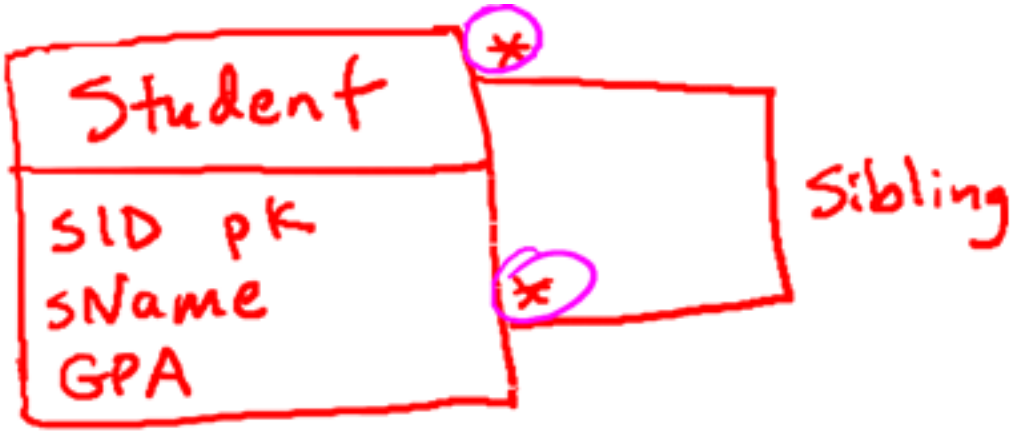
College(cName, ...)

Applied(sID, cName, date, decision)
PK: (sID, cName)
FK: sID → Student
FK: cName → College
```



UML to Relations

Self-Associations

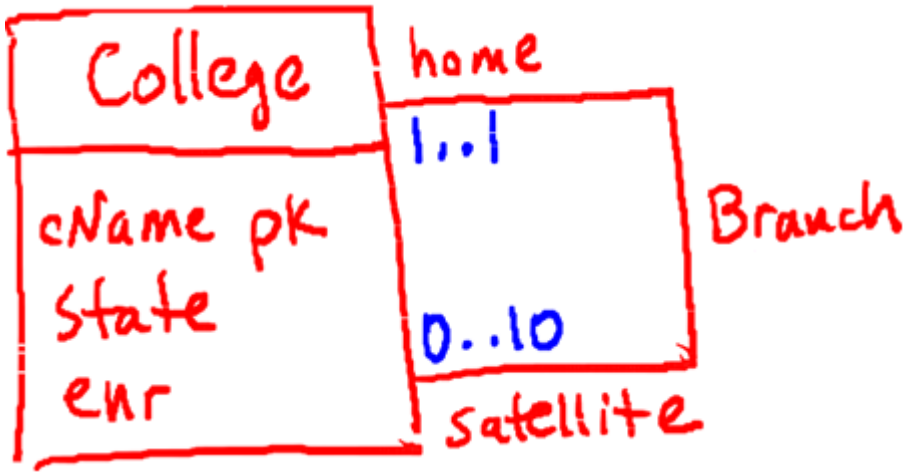


Student (sID, sName, GPA)

Sibling (sID1, sID2)

UML to Relations

Self-Associations



College (cName, state, enr)

Branch (home, satellite)

UML to Relations

3-Possible Translations

Design-1: Super relation + Subclass relations contain superclass key + specialized attributes (**Table per class**).

Design-2: Subclass relations contain all attributes (**Table per concrete class**).

Design-3: One relation containing all superclass + subclass attributes (**One single table**).



Possible relation designs:

1) S(<u>K</u>,A), S1(<u>K</u>,B), S2(<u>K</u>,C)
2) S1(<u>K</u>,A,B), S2(<u>K</u>,A,C)
3) S(<u>K</u>,A,B,C)

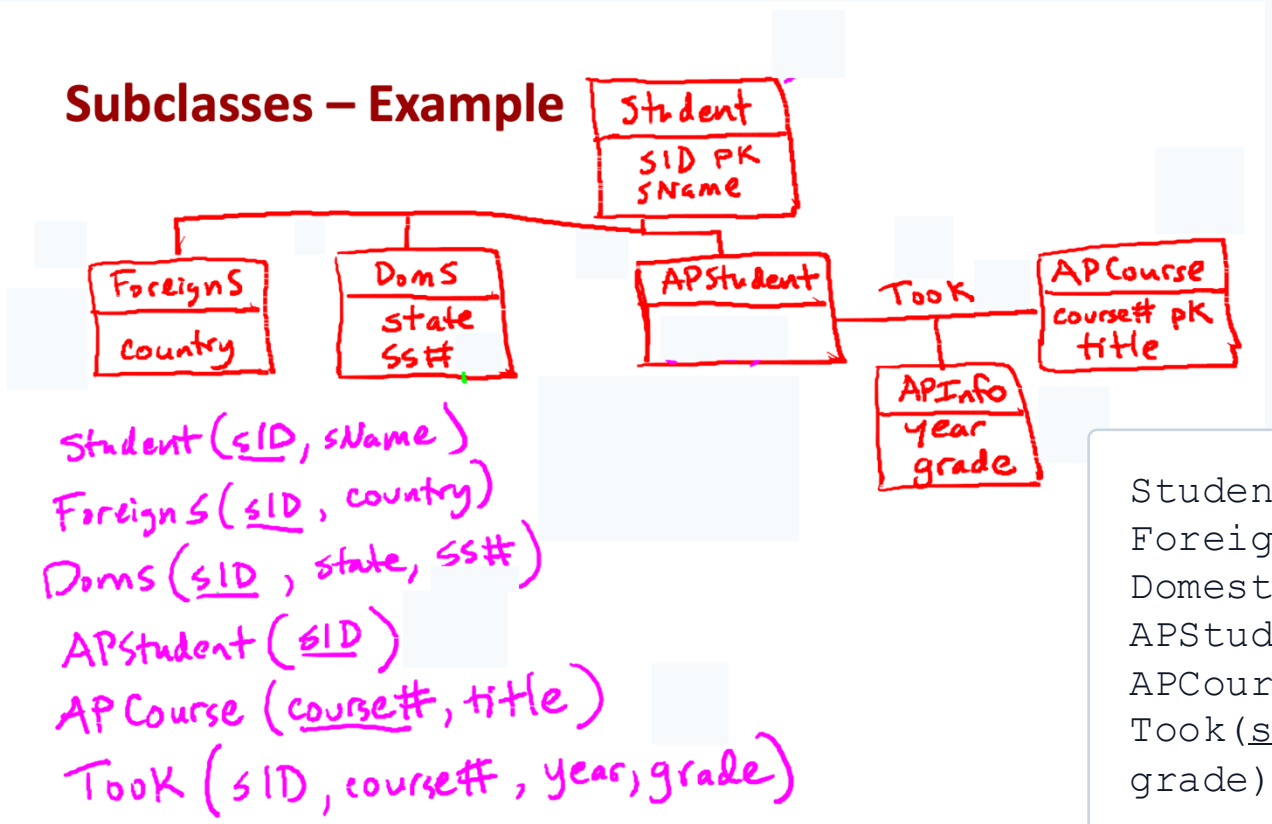
Best translation depends on on complete, joint, overlap, and query workload.

UML to Relations

Design 1: table per class

- Superclass table stores shared attributes.
- Each subclass table stores superclass key + specialized attributes.
- Most normalized; often requires joins to get complete object information.

```
S(K, A)
S1(K, B)
S2(K, C)
```




```
Student(sID, sName)
Foreign(sID, country)
Domestic(sID, state, SSN)
APStudent(sID)
APCourse(course#, title)
Took(sID, course#, year,
grade)
```

UML to Relations

Design 2: table per concrete class

- No separate superclass table.
- Each subclass table contains inherited + specialized attributes.
- Good for complete/disjoint subclasses or abstract superclass.
- Can duplicate superclass attributes.

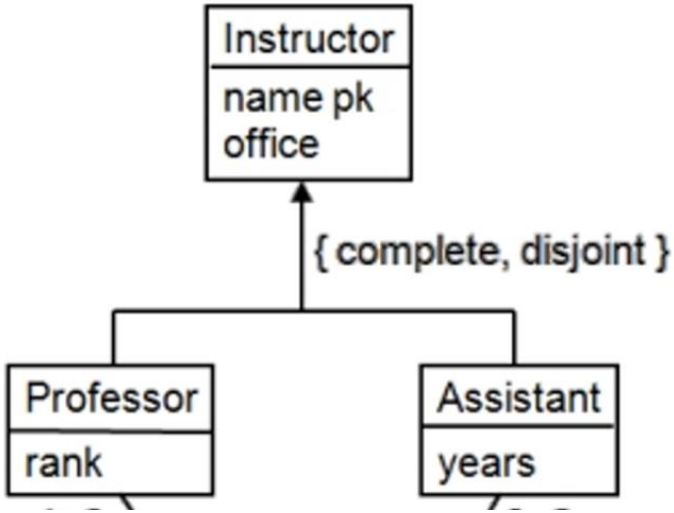
```
S1(K, A, B)
S2(K, A, C)
```

Design 3: single table

- One relation contains all superclass and subclass attributes.
- Simple to query.
- May contain many NULL values for attributes that do not apply.
- Works better when subclasses overlap heavily.

```
S(K, A, B, C)
```

UML to Relations: Example



- Best Translation: Design 2 (Table per Concrete Class)
- Each subclass table contains ALL attributes (inherited + specialized)
- Professor Table (includes all inherited attributes from Instructor)

```
CREATE TABLE Professor (
    name VARCHAR(100) PRIMARY KEY,
    office VARCHAR(50),
    rank VARCHAR(20)
);
```

- Assistant Table (includes all inherited attributes from Instructor)
- No separate Instructor table needed since it's abstract

```
CREATE TABLE Assistant (
    name VARCHAR(100) PRIMARY KEY,
    office VARCHAR(50),
    years INTEGER
);
```

Instructor is abstract, complete, and disjoint.
Professor and Assistant are concrete subclasses.
A table-per-concrete-class design is simple here.

Professor(name, office, rank)
Assistant(name, office, years)

UML to Relations

Composition & Aggregation



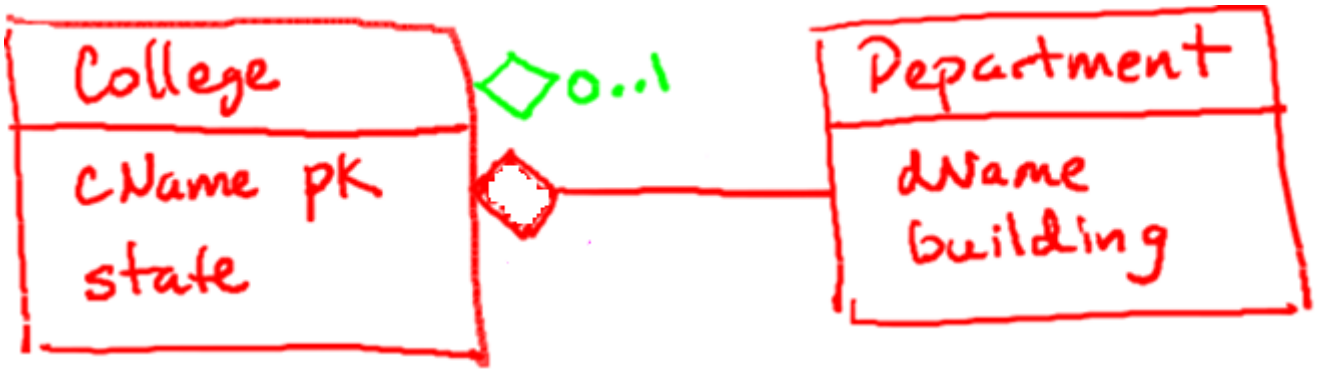
Department is a Component of College

College(cName, state)

Department(dName, building, cName)

UML to Relations

Composition & Aggregation



College(cName, state)

Department(dName, building, cName)

College can be NULL

UML for designing your database

UML: High-Level Database Design Model

- User-friendly graphical specification language
- Designs translated to relations automatically

*** Exercise-V: UML to Relations ***